

AE Latent Vector 거리·각도 분석 파이프라인의 구조 개선 및 성능 최적화

26-1 고급파이썬프로그래밍및자동화] 최종 보고서

202551130 최서연

1. 연구 코드 소개

개선 대상 코드 : `ae_s919_original_variant_metrics.py`

해당 코드는 일명 Sphere 프로젝트에서 사용하는 코드로, ImageNetS919 데이터셋의 원본 이미지와 여러 형태로 변형된 variant 이미지들을 Autoencoder encoder에 통과시켜 latent vector를 생성하고, 원본 이미지의 latent vector와 variant 이미지의 latent vector 사이의 거리 및 각도 변화를 정량적으로 계산하는 역할을 한다.

[연구 주제 또는 실험 맥락]

Sphere 프로젝트의 핵심 목표는 이미지의 시각적 변형이 모델의 latent representation 공간에서 어떤 방식으로 반영되는지를 분석하는 것이다. 원본 이미지와 여러 변형들을 적용한 배경 변경, 마스크 적용, 노이즈 추가, 투명 배경, 흑백 합성 등의 이미지 사이의 차이가 Autoencoder의 latent space에서 얼마나 크게 나타나는지를 비교하고자 한다.

본 코드에서는 ImageNetS919 single-object 이미지들을 대상으로 하며, 각 이미지에 대해 원본 이미지와 여러 variant 이미지를 쌍으로 구성한다. 이후 사전학습된 ImageNet Autoencoder의 encoder를 사용하여 각 이미지를 latent vector로 변환하고, 원본 latent vector와 variant latent vector 사이의 L2 distance, cosine similarity, cosine distance, angle radian, norm difference, relative L2 등을 계산한다. 이를 통해 이미지 변형 방식별로 latent space에서의 변화량을 비교할 수 있다. 따라서, 본 코드는 Sphere 프로젝트에서 이미지 변형이 표현 공간에 미치는 영향을 정량적으로 분석하기 위한 핵심 실험 코드이다.

[해당 코드의 입력과 출력]

- 입력

1. metadata CSV 파일 : ImageNetS919 이미지의 `class_id`, `image_id`, `original_path`, foreground label 수 등의 정보가 포함되어 있다.
코드는 이 metadata를 읽은 뒤 foreground label이 하나인 이미지, 즉 single-object 이미지에 해당하는 행만 필터링한다.
2. 원본 이미지와 variant 이미지 파일 : 원본 이미지는 metadata의 `original_path`에서 가져오고, variant 이미지는 variant 종류에 따라 별도의 디렉터리에서 로드한다. 수행한 변형 종류는 `alpha`, `bg_color`, `bg_inpaint`, `bg_transparent`, `comp_black`, `comp_white`, `cutout_rgba`, `gaussian`, `masked_black`, `masked_white`, `salt_pepper`, `mask_white`로, 각 설명은 다음과 같다.

Variant	설명
alpha	[투명도/alpha 계열] 원본 객체 또는 마스크 영역에 투명도 계수 적용해 시각적 강도 조절한 이미지
bg_color	[배경 변형 계열] 객체는 유지하고 배경을 특정 단색 또는 색상 조건으로 대체한 이미지
bg_inpaint	[배경 변형 계열] 객체 주변 또는 제거 영역의 배경을 inpainting 방식으로 복원하거나 대체한 이미지
bg_transparent	[배경 변형 & 투명도/alpha 계열] 배경을 투명하게 만든 RGBA 기반 이미지
comp_black	[객체 합성 계열] 객체 또는 foreground를 검은색 배경 위에 합성한 이미지
comp_white	[객체 합성 계열] 객체 또는 foreground를 흰색 배경 위에 합성한 이미지
cutout_rgba	[객체 합성 & 투명도/alpha 계열] 객체 영역만 잘라내고 나머지 영역을 alpha channel로 투명 처리한 cutout 이미지
gaussian	[노이즈 계열] 원본 또는 특정 영역에 Gaussian noise를 추가한 이미지
salt_pepper	[노이즈 계열] 흰 점 / 검은 점 형태의 salt-and-pepper-noise를 추가한 이미지
masked_black	[마스크 기반 계열] 마스크 영역을 검은색으로 처리한 이미지
masked_white	[마스크 기반 계열] 마스크 영역을 흰색으로 처리한 이미지
mask_white	[마스크 기반 계열] 객체 마스크를 흰색으로 표현한 마스크 기반 이미지

3. 사전학습된 Autoencoder checkpoint : 코드에서는 외부 ImageNet Autoencoder repository를 불러오고, `imagenet-vgg16.pth` checkpoint를 로드한 뒤 encoder 부분을 사용하여 이미지를 latent vector로 변환한다.

- 출력

1. 각 원본-variant pair에 대한 metric CSV 파일 : 이미지 식별자, variant 종류, 원본 이미지 경로, variant 이미지 경로, latent vector shape, latent dimension, L2 distance, cosine similarity, cosine distance, angle radian, norm original, norm variant, norm difference, relative L2 등이 저장된다.
2. 원본 이미지와 각 변형 이미지들의 latent vector 파일 : 각 latent vector는 .npy 파일로 저장된다. 이를 통해 후속 분석에서 이미지를 다시 encoder에 통과시키지 않고 저장된 latent vector를 재사용할 수 있다.
3. variant별 summary CSV 파일 : 전체 metric 결과를 variant 기준으로 groupby하여 평균, 표준편차, 중앙값, 최솟값, 최댓값을 계산한다. 이를 통해 어떤 변형 방식이 latent space에서 더 큰 변화를 유발하는지 비교할 수 있다.
4. 전체 metric에 대한 summary CSV 파일 : variant 구분 없이 전체 결과의 평균, 표준편차, 중앙값, 최솟값, 최댓값을 계산하여 실험 전체의 분포를 요약한다.

[전체 연구 파이프라인에서의 위치]

- 전체 Sphere 연구 파이프라인에서 해당 코드는 이미지 변형 생성 단계 이후, latent representation 분석 단계에 위치한다.

전체 흐름은 다음과 같이 정리할 수 있다.

1. 원본 ImageNetS919 이미지와 annotation metadata를 준비한다.
2. segmentation 또는 mask 정보를 이용하여 single-object image를 선별한다.
3. 원본 이미지에 대해 배경 변경, 마스킹, 노이즈 추가, 투명화 등의 variant 이미지를 생성한다.
4. 본 코드에서 원본 이미지와 variant 이미지를 Autoencoder encoder에 입력하여 latent vector를 생성한다.
5. 원본 latent vector와 variant latent vector 사이의 거리 및 각도 기반 metric을 계산한다.
6. variant별 summary를 생성하여 변형 방식에 따른 latent space 변화량을 비교한다.
7. 후속 분석 또는 시각화 단계에서 각 variant가 representation에 미치는 영향을 해석한다.

따라서, 본 코드는 변형이 Autoencoder latent space에서 어느 정도의 수치적 변화를 유발하는지를 측정하는 핵심 분석 단계에 해당하며, 이 단계의 결과는 이후 variant 간 비교, latent geometry 분석, representation robustness 분석 등에 활용될 수 있다.

[해당 코드를 개선 대상으로 선택한 이유]

1. 동일한 원본 이미지가 여러 variant와 반복적으로 비교되기 때문에 중복 계산이 발생할 가능성이 크다. 현재 코드는 원본-variant pair를 처리 단위로 삼고 있으므로, 하나의 원본 이미지가 12개 variant와 비교될 경우 원본 이미지의 latent vector가 반복적으로 계산될 수 있다. Autoencoder encoder 호출은 GPU 연산과 이미지 전처리를 포함하므로 전체 실행 시간에서 큰 비중을 차지한다. 따라서 original latent vector를 캐싱하거나 재사용하는 방식으로 개선하면 실질적인 성능 향상을 기대할 수 있다.
2. 연구 파이프라인의 규모가 커질수록 pair 수가 빠르게 증가한다. 이미지 수가 N개이고 variant 수가 V개라면 원본-variant pair 수는 대략 $N \times V$ 개가 된다. 이때 모든 pair를 list로 구성하고, 각 pair마다 이미지 로드, 전처리, encoding, metric 계산, CSV 저장을 수행하면 실행 시간과 메모리 사용량이 증가한다. 따라서 generator 또는 iterator 기반 처리 구조, lazy evaluation, 중간 materialization 감소와 같은 수업 개념을 적용하기에 적합하다.
3. 현재 코드는 경로 설정, 모델 로드, 이미지 전처리, latent 생성, metric 계산, latent 저장, progress 저장, summary 생성이 하나의 스크립트에 함께 들어 있다. 이 구조는 실험을 빠르게 실행하기에는 편리하지만, 실험 조건을 바꾸거나 benchmark를 수행하거나 일부 기능을 재사용하기에는 유지보수성이 떨어질 수 있다. 따라서 dataclass 기반 config 분리, metric 계산 함수 분리, timing decorator 도입, caching 정책 분리 등을 적용하면 코드의 구조적 명확성과 재사용성을 높일 수 있다.

결론적으로 이 코드는 Sphere 프로젝트의 실제 연구 분석에 사용되는 코드이며, Autoencoder latent vector 생성과 거리·각도 계산이라는 명확한 연구 목적을 가진다. 동시에 반복 encoding, pair 단위 처리, 전역 설정값, 단일 스크립트 중심 구조 등 최적화와 리팩터링이 가능한 지점이 뚜렷하다. 그렇기 때문에 본 과제의 목표인 Python 자료구조, iterator/generator, class / dataclass, decorator, caching, benchmark 기반 코드 개선을 적용하기에 적절한 대상 코드라고 판단하였다.

2. 기존 코드의 문제점 분석

Sphere 프로젝트의 `ae_s919_original_variant_metrics.py` 코드는 원본 이미지와 variant 이미지를 쌍으로 구성하고, 각 pair에 대해 Autoencoder encoder를 이용해 latent vector를 생성한 뒤 거리 및 각도 기반 metric을 계산한다. 다만, 전체적인 기능은 명확하더라도, 연구 데이터 규모가 커질수록 중복 연산, 불필요한 materialization, 설정 관리의 어려움, 함수 책임 과다 등의 문제가 발생할 수 있다.

2.1. 중복 original encoding 문제 :

가장 큰 성능 병목은 동일한 원본 이미지의 latent vector를 variant 수만큼 반복 계산한다는 점이다. 현재 코드는 원본 이미지 단위가 아니라 원본-variant pair 단위로 처리되며, 여러 variant들에 대해서 같은 원본 이미지가 각 변형과의 비교를마다 다시 로드되고 다시 encoder에 입력된다.

현재 처리 흐름은 개념적으로 다음과 같다.

```
for row in valid_rows:
    original_img = load_pil_rgb(row["original_path"])
    variant_img = load_pil_rgb(row["variant_path"])

    z_orig = encode_image(encoder, transform, original_img, device)
    z_var = encode_image(encoder, transform, variant_img, device)

    metric_dict = compute_metrics(z_orig, z_var)
```

이 구조에서는 원본 이미지 수를 N , variant 수를 V 라고 할 때, 원본 latent encoding 횟수가 N 이 아니라 $N \times V$ 에 가깝게 증가한다. 예를 들어 하나의 이미지에 12개 variant가 존재하면, 원본 이미지는 이론적으로 한 번만 encoding해도 충분하지만 현재 구조에서는 최대 12번 반복 encoding될 수 있다.

Autoencoder encoder 호출은 이미지 resize, center crop, tensor 변환, GPU 전송, encoder forward pass를 포함하여 단순한 Python 연산보다 비용이 훨씬 크다. 특히 GPU inference가 포함된 연구 코드에서는 encoder 호출 횟수가 전체 실행 시간에 직접적인 영향을 미친다. 그렇기 때문에 original latent vector를 (`class_id`, `image_id`) 또는

`original_path` 기준으로 cache하거나, 이미 저장된 `.npy` latent 파일을 재사용하는 방식이 필요하다.

알고리즘의 처리 단위를 현재 코드에서는 pair 중심으로 설계하였으나, 실제로는 원본 이미지는 image 단위로 한 번 encoding하고, variant 이미지만 variant 단위로 encoding하는 구조가 더 적절할 것이다.

2.2. Pair list materialization 문제 :

현재 코드는 `build_pair_rows()` 함수에서 metadata CSV를 읽고, 각 이미지와 variant 조합에 대한 row를 모두 list에 저장한다. 이후 `main()` 함수에서는 이 전체 `rows` list를 다시 순회하여 `valid_rows`와 `missing_rows`를 별도의 list로 생성한다.

개념적으로 다음과 같은 구조이다.

```
rows = build_pair_rows()
valid_rows = [r for r in rows if r["original_exists"] and r["variant_exists"]]
missing_rows = [r for r in rows if not (r["original_exists"] and r["variant_exists"])]
```

현재의 방식은 데이터 규모가 작을 때는 구현이 단순하다는 장점이 있으나, 이미지 수와 variant 수가 증가하면 pair 수는 $N \times V$ 로 증가한다. 모든 pair 정보를 Python dictionary 형태로 list에 저장하고, 다시 valid/missing list로 나누면 동일한 metadata가 여러 list에 중복 보관된다는 문제가 생기고, 이는 수업에서 다룬 eager materialization의 전형적인 예라 할 수 있다.

한 번에 하나의 pair 또는 batch 단위 pair만 있어도 수행할 수 있기 때문에, 전체 pair를 먼저 memory에 고정한 뒤 처리하지 않고, pair row를 generator로 순차 생산하거나, valid/missing 판정을 처리 루프 내부에서 수행하면 중간 list 생성을 줄일 수 있다.

물론, 현재 코드에서는 `pairs_all.csv`, `missing_pairs.csv`, `progress.json` 등을 생성해야 하므로 pair 정보를 완전히 streaming 처리하는 방식에는 설계상의 trade-off가 있지만, 적어도 분석 루프에서는 모든 pair를 별도의 `valid_rows` list로 materialize하지 않고, iterator 또는 generator 기반으로 필요한 row를 순차 처리하는 구조가 더 확장성 있을 것이다.

2.3. 전역 설정값 관리 문제 :

현재 코드에서는 경로, 모델 설정, 이미지 전처리 크기, device, seed, variant 목록, latent 저장 여부 등이 모두 전역 상수로 정의되어 있다.

예를 들어 다음과 같은 값들이 스크립트 상단에 직접 선언되어 있다.

```
META_PATH = Path(...)
VARIANT_ROOT = Path(...)
OUT_DIR = Path(...)
LATENT_DIR = OUT_DIR / "latents"

EXTERNAL_REPO_ROOT = Path(...)
AE_CHECKPOINT_PATH = Path(...)
ARCH = "vgg16"

DEVICE = "cuda"
RESIZE_SIZE = 256
CROP_SIZE = 224
SEED = 42
VARIANTS = [...]
SAVE_LATENTS = True
```

이 구조는 단일 실험을 빠르게 실행할 때는 편리하나 실험 조건이 바뀌거나 benchmark를 수행해야 할 때는 불리하다. 예를 들어 crop size를 바꾸거나, latent 저장 여부를 끄거나, output directory를 benchmark 전용 경로로 바꾸려면 코드 내부 상수를 직접 수정해야 한다. 이는 실험 재현성을 떨어뜨리고, before/after 성능 비교를 어렵게 만든다.

또한 설정값이 전역 변수로 흩어져 있으면 함수 간 의존성이 명시적으로 드러나지 않는다. 예를 들어 `variant_path_for()`는 `VARIANT_ROOT`와 `PNG_VARIANTS`라는 전역 상태에 의존하고, `latent_path_for()`는 `LATENT_DIR`에 의존한다. 이 경우 함수를 다른 실험이나 테스트 코드에서 재사용하기 어렵다.

따라서 수업 내용에 따라 설정값은 `dataclass` 기반의 config 객체로 묶는 것이 적절할 것이다. 예를 들어 `LatentAnalysisConfig` 안에 metadata 경로, variant root, output directory, checkpoint path, resize size, crop size, device, save option 등을 포함하면 실험 조건을

하나의 객체로 명시할 수 있다. 특히 `frozen=True` 옵션을 사용하면 실험 도중 config가 임의로 변경되는 것을 막을 수 있어 재현성 측면에서도 유리하다.

2.4. `main()`함수의 책임 과다 문제 :

현재 `main()` 함수는 전체 파이프라인의 거의 모든 책임을 담당한다. 구체적으로는 다음과 같은 작업이 모두 하나의 함수 안에서 수행된다.

1. output directory 생성
2. device 선택
3. pair row 생성
4. valid/missing pair 분리
5. pair metadata CSV 저장
6. image transform 생성
7. AE encoder 로드
8. resume progress 확인
9. CSV writer 초기화
10. 이미지 로드
11. latent vector 생성
12. metric 계산
13. latent 파일 저장
14. metric row 저장
15. progress JSON 저장
16. summary CSV 생성

이 구조는 절차적으로는 이해하기 쉬울 수는 있지만, 하나의 함수가 파일 시스템, 모델 로딩, 데이터 순회, GPU inference, metric 계산, 예외 처리, progress 관리, summary 계산을 모두 담당하기 때문에 갖는 책임이 지나치게 많고, 코드 수정의 영향 범위가 커진다.

예를 들어 metric 계산 방식만 바꾸고 싶어도 `main()` 내부 흐름을 함께 이해해야 하며, benchmark를 위해 encoding 구간과 metric 계산 구간을 따로 측정하려면 기존 함수 구조를 다시 분해해야 한다. 또한 `main()`을 실행하면 실제 파일 경로, 실제 checkpoint, 실제 GPU 환경에 강하게 의존하기 때문에 단위 테스트를 작성하기도 어렵다.

따라서 다음과 같이 책임을 분리할 필요가 있다.

: config 로드 / pair row 생성 / encoder 로드 / latent encoding / metric 계산 /
result writing / progress 저장 / summary 생성 / benchmark 측정

이처럼 역할을 나누면 각 함수가 단일한 책임을 가지게 되고, 특정 단계만 교체하거나 측정하기 쉬워질 것이다. 이는 객체지향 설계에서 말하는 단일 책임 원칙과도 연결된다.

2.5. Metric 계산의 중복 가능성 :

현재 `compute_metrics()` 함수는 원본 latent와 variant latent의 차이를 기반으로 여러 metric을 계산한다. 이 함수는 별도로 분리되어 있어 구조적으로는 좋은 편이다. 그러나 내부 계산에는 일부 중복 가능성이 있다.

현재 계산 항목은 다음과 같다.

```
diff = z_var - z_orig
l2 = torch.linalg.norm(diff).item()
norm_original = torch.linalg.norm(z_orig).item()
norm_variant = torch.linalg.norm(z_var).item()

cos = F.cosine_similarity(z_orig.unsqueeze(0), z_var.unsqueeze(0), dim=1).item()
cosine_distance = 1.0 - cos
angle_radian = np.arccos(cos)
norm_difference = norm_variant - norm_original
relative_l2 = l2 / max(norm_original, 1e-12)
```

여기서 cosine similarity는 내부적으로 두 vector의 dot product와 norm을 사용하는데, 코드에서는 이미 `norm_original`과 `norm_variant`를 별도로 계산하고 있기 때문에 cosine similarity를 `F.cosine_similarity()`에 다시 위임하면 norm 관련 계산이 중복될 수 있다.

물론 이 중복 계산의 비용은 Autoencoder encoding 비용에 비하면 상대적으로 작으나 latent vector 수가 매우 많아지고 metric 계산만 반복 수행하는 후속 분석 단계에서는 불필요한 계산이 누적될 수 있다. 따라서 dot product, norm original, norm variant를 한 번 계산하고 이를 cosine similarity, cosine distance, angle 계산에 함께 사용하는 방식이 더 명시적이고 효율적이다.

예를 들어 다음과 같이 계산할 수 있다.

```
dot = torch.dot(z_orig, z_var).item()
norm_original = torch.linalg.norm(z_orig).item()
norm_variant = torch.linalg.norm(z_var).item()
cos = dot / max(norm_original * norm_variant, 1e-12)
cos = float(np.clip(cos, -1.0, 1.0))
```

이 방식은 수학적으로 각 metric이 어떤 중간값에 의존하는지 더 명확하게 드러낸다는 장점이 있다.

2.6. I/O와 연산의 결합 문제 :

현재 코드는 이미지 로드, latent 저장, CSV 기록, progress 저장 같은 I/O 작업과 Autoencoder encoding, metric 계산 같은 연산 작업이 하나의 루프 안에 결합되어 있다.

처리 루프 안에서는 PIL image load / image transform / GPU tensor transfer / encoder forward / latent CPU transfer / metric calculation / latent npy save / CSV row write / progress JSON write 작업들이 연속적으로 수행된다.

이 구조는 실행 흐름을 한눈에 보기에는 쉽지만, 성능 분석에는 불리하다. 전체 실행 시간이 길어졌을 때 원인이 이미지 디스크 I/O인지, GPU inference인지, latent 저장인지, CSV flush인지 분리해서 파악하기 어렵다. 또한 저장 옵션을 끄고 순수 encoding 성능만 측정하거나, 이미 저장된 latent를 사용하여 metric 계산만 수행하는 실험을 구성하기 어렵다.

특히 이 코드에서는 `SAVE_LATENTS`가 활성화된 경우 `.npy` 파일 저장이 pair마다 수행된다. variant latent는 매번 저장되고, original latent는 파일이 없을 때만 저장된다. 그러나 original latent 파일이 이미 존재하더라도 현재 루프에서는 original 이미지를 다시 로드하고 다시 encoding한 뒤, 저장만 생략한다. 즉, disk cache가 저장 단계에만 반영되고 계산 단계에는 반영되지 않는다.

따라서 I/O와 연산을 분리하여 다음과 같은 구조로 개선할 필요가 있다.

1단계: latent가 이미 존재하는지 확인

2단계: 존재하면 disk cache에서 로드

3단계: 없으면 이미지를 로드하고 encoder로 계산

4단계: 계산된 latent를 저장

5단계: metric 계산은 latent tensor만 입력으로 받아 수행

이렇게 분리하면 저장된 latent를 실제 cache로 활용할 수 있고, 이미지 로딩 및 encoder forward의 중복을 줄일 수 있다.

2.7. Resume 및 progress 처리의 구조적 한계 :

현재 코드는 `progress.json`에 `processed_pairs` 값을 저장하고, 재실행 시 해당 index 이전의 row는 건너뛰는 방식으로 resume 기능을 구현한다. 이 방식은 단일 실행이 중단되었을 때 이어서 실행할 수 있다는 장점이 있다.

그러나 현재 resume 방식은 `valid_rows`의 순서가 이전 실행과 완전히 동일하다는 가정에 의존한다. 만약 metadata CSV가 바뀌거나, variant 목록이 바뀌거나, missing file 상태가 바뀌면 동일한 index가 동일한 pair를 의미하지 않을 수 있다. 즉, index 기반 progress는 데이터셋 구성이 바뀌는 상황에서 안정성이 떨어진다.

또한 현재 구조에서는 row 처리 도중 예외가 발생해도 루프는 계속 진행된다. 이때 실패한 row는 warning으로 출력되지만, 실패 목록이 별도의 구조화된 CSV로 저장되지는 않는다. 따라서 후속 분석에서 어떤 pair가 실패했고, 어떤 오류가 발생했는지를 재현 가능하게 추적하기 어렵다.

progress 저장 시점도 100개 row마다 한 번씩 이루어진다. 이는 I/O 빈도를 줄인다는 장점은 있지만, 중간에 실행이 종료되면 마지막 progress 저장 이후 처리된 row들은 다시 계산될 수 있다. 이 자체는 큰 문제는 아니지만, encoder 호출 비용이 큰 코드에서는 재계산 비용이 누적될 수 있다.

더 안정적인 방식은 pair index만 저장하는 것이 아니라, (`class_id`, `image_id`, `variant`)를 기준으로 처리 완료 여부를 기록하는 것이다. 예를 들어 결과 CSV에 이미 존재하는 pair key를 set으로 읽어온 뒤, 재실행 시 해당 key가 있으면 건너뛰는 방식이 더 안전하다. 이 방식은 row 순서가 바뀌어도 이미 처리된 pair를 식별할 수 있다.

문제점 요약

기존 코드는 연구 목적에 필요한 전체 기능을 수행하지만, 성능 최적화와 유지보수성 측면에서는 다음과 같은 한계를 가진다.

첫째, 원본 이미지의 latent vector가 variant마다 반복 계산되어 Autoencoder encoder 호출 횟수가 불필요하게 증가한다. 둘째, 전체 pair 정보를 list로 materialize하고 다시 valid/missing list로 분리하므로 데이터 규모가 커질수록 메모리 사용량과 중간 객체 생성 비용이 증가한다. 셋째, 실험 설정값이 전역 상수로 흩어져 있어 실험 조건 변경과 benchmark 구성이 어렵다. 넷째, `main()` 함수가 모델 로드, 데이터 순회, encoding, metric 계산, 저장, progress 관리까지 모두 담당하여 단일 책임 원칙을 위반한다. 다섯째, metric 계산에서 norm과 cosine 관련 계산이 중복될 가능성이 있다. 여섯째, 이미지/latent/CSV I/O와 GPU 연산이 하나의 루프에 결합되어 있어 병목 분석과 단계별 benchmark가 어렵다. 마지막으로, index 기반 resume 방식은 metadata나 variant 구성이 바뀌는 경우 안정성이 떨어질 수 있다.

따라서 본 코드의 개선 방향은 original latent caching, generator 기반 pair 처리, dataclass 기반 설정 분리, decorator 기반 timing/logging, metric 계산 정리, I/O와 연산 단계 분리, key 기반 resume 방식 도입으로 설정할 수 있다. 이러한 개선은 단순한 코드 정리가 아니라, Sphere 프로젝트의 latent representation 분석 파이프라인을 더 빠르고 재현 가능하며 확장 가능한 구조로 만드는 데 목적이 있다.

3. 적용한 수업 개념

3.1. 중복 original encoding 문제 → dict 기반 caching, PyTorch inference wrapper

7.2.1에서 확인한 가장 큰 문제는 동일한 original image가 여러 variant와 비교될 때마다 반복적으로 encoding된다는 점이었다. 기존 코드는 원본-variant pair를 처리 단위로 삼기 때문에, 하나의 원본 이미지에 12개 variant가 존재하면 original latent vector가 최대 12번 반복 계산될 수 있다.

이를 해결하기 위해 dict 기반 original latent cache 개념을 적용하였다. (`class_id`, `image_id`) 또는 `original_path`를 key로 사용하고, 한 번 계산한 original latent vector를 cache에 저장한 뒤 같은 original image가 다시 등장하면 encoder를 다시 호출하지 않고 cache에서 latent vector를 재사용한다.

이때 dict를 선택한 이유는 key 기반 탐색이 필요하기 때문이다. list를 사용하면 특정 original image의 latent가 이미 계산되었는지 확인하기 위해 선형 탐색이 필요하지만, dict를 사용하면 image identifier를 기준으로 직접 접근할 수 있다. 따라서 original latent 재사용 문제에는 dict가 가장 적절한 자료구조이다.

또한 이 코드는 학습 코드가 아니라 사전학습된 Autoencoder encoder를 이용한 inference 코드이므로, `torch.no_grad()`와 `model.eval()`을 명시적으로 유지하였다. 이는 autograd graph 생성을 방지하여 불필요한 메모리 사용을 줄이고, inference 목적에 맞는 실행 모드를 보장하기 위한 PyTorch inference wrapper 개념에 해당한다.

3.2. Pair list materialization 문제 → iterator/generator, lazy evaluation

7.2.2에서 지적한 문제는 `build_pair_rows()`가 모든 원본-variant pair를 list로 먼저 생성하고, 이후 다시 `valid_rows`, `missing_rows`를 별도 list로 분리한다는 점이었다. 이미지 수를 N , variant 수를 V 라고 할 때 pair 수는 $N \times V$ 로 증가하므로, 모든 pair를 먼저 materialize하는 구조는 데이터 규모가 커질수록 중간 객체 생성 비용과 메모리 사용량을 증가시킨다.

이를 해결하기 위해 iterator와 generator 개념을 적용하였다. 수업에서 iterator는 “다음 값을 생산할 수 있는가”라는 반복의 최소 계약으로 설명되었고, generator는 이 계약을 함수 형태로 간결하게 구현하는 방법으로 다루었다. 이 코드에서 pair row는 전체 list로 먼저 고정될 필요가 없고, metadata와 variant 목록으로부터 하나씩 생성되어 바로 소비될 수 있다.

따라서 `build_pair_rows()`를 모든 row를 반환하는 함수가 아니라 `iter_pair_rows()`처럼 row를 하나씩 `yield`하는 generator로 변경한다. 분석 루프는 이 generator를 소비하면서 존재하는 pair는 metric 계산으로 넘기고, missing pair는 별도 기록한다.

이 변경은 기존의 eager evaluation 구조를 lazy evaluation 구조로 바꾸는 것이다. 다만 모든 단계에서 lazy evaluation을 강제하지는 않았다. variant별 summary를 계산할 때는 전체 metric 결과를 pandas DataFrame으로 읽어 집계해야 하므로, 이 단계에서는 eager materialization이 적절하다. 즉, pair 처리 단계는 lazy하게 유지하고, 통계 요약 단계는 DataFrame으로 materialize하는 방식으로 evaluation 경계를 구분하였다.

3.3. 전역 설정값 관리 문제 → dataclass 기반 config 분리

7.2.3에서 확인한 문제는 metadata 경로, variant root, output directory, checkpoint path, device, resize size, crop size, seed, variant 목록, latent 저장 여부 등이 모두 전역 상수로 흩어져 있다는 점이었다. 이 구조에서는 실험 조건을 바꾸거나 benchmark용 설정을 구성할 때 코드 본문을 직접 수정해야 한다.

이를 해결하기 위해 `dataclass` 기반 `config` 분리를 적용하였다.

`LatentAnalysisConfig` 객체 안에 실험에 필요한 경로, 모델 정보, 이미지 전처리 설정, 저장 옵션, `variant` 목록을 묶는다. 이렇게 하면 함수들이 전역 변수에 암묵적으로 의존하지 않고, 명시적으로 `config` 객체를 입력받아 동작할 수 있다.

특히 `frozen=True`를 사용하면 실험 실행 중 `config` 값이 임의로 변경되는 것을 막을 수 있다. 이는 연구 코드의 재현성과 직접 연결된다. 어떤 `metadata`, 어떤 `checkpoint`, 어떤 `resize/crop` 설정, 어떤 `variant` 목록으로 `latent`를 생성했는지가 `config` 객체에 명확히 남기 때문이다.

3.4. `main()` 함수 책임 과다 문제 → 단일책임원칙, `class/function` 역할 분리

7.2.4에서 분석한 것처럼 기존 `main()` 함수는 `directory` 생성, `device` 선택, `pair` 생성, `encoder` 로드, `resume` 처리, `CSV writer` 생성, 이미지 로드, `latent` 생성, `metric` 계산, `latent` 저장, `progress` 저장, `summary` 생성까지 거의 모든 책임을 담당한다. 이는 단일책임원칙을 위반하는 구조이다.

이를 해결하기 위해 단일책임원칙을 적용하였다. 전체 파이프라인을 하나의 거대한 함수에 두지 않고, `config` 관리, `pair` 생성, `encoder` 로드, `latent` 생성 또는 로드, `metric` 계산, 결과 기록, `progress` 관리, `summary` 생성으로 역할을 분리한다.

이 분리는 단순히 코드를 여러 함수로 쪼개는 것이 아니라, 각 함수가 하나의 명확한 책임을 갖도록 만드는 것이다. 예를 들어 `compute_metrics()`는 `latent tensor` 두 개를 받아 `metric dictionary`만 반환하고, `get_original_latent()`는 `original latent`를 `cache`에서 가져오거나 새로 `encoding`하는 책임만 갖는다. `run_pipeline()`은 각 단계의 실행 순서를 조율하는 `orchestration` 역할만 담당한다.

이렇게 분리하면 특정 단계만 `benchmark`하거나, `cache` 정책만 바꾸거나, `metric` 계산 방식만 교체하는 것이 쉬워진다. 또한 실제 경로와 `GPU` 환경에 강하게 묶여 있던 `main()` 중심 구조보다 테스트와 재사용이 용이해진다.

3.5. `Metric` 계산 중복 가능성 → 중간값 재사용과 계산 구조 명시화

7.2.5에서 지적한 문제는 `compute_metrics()` 내부에서 `norm`을 별도로 계산하면서, `cosine similarity` 계산을 `F.cosine_similarity()`에 다시 위임한다는 점이었다. `cosine`

similarity는 내부적으로 dot product와 norm을 사용하므로, 이미 계산한 norm을 재사용하지 못하는 구조이다.

이를 해결하기 위해 dot product, original norm, variant norm, diff norm을 명시적으로 계산하고, 이 중간값을 여러 metric 계산에 재사용하도록 수정한다. 이 개념은 자료구조 최적화보다는 계산 중복 제거와 계산 흐름 명시화에 가깝다.

이 변경의 성능 개선 폭은 Autoencoder encoding cache에 비해 작을 수 있다. 하지만 metric 계산이 latent vector 수만큼 반복되는 후속 분석에서는 중복 계산이 누적될 수 있고, 무엇보다 각 metric이 어떤 수학적 중간값에 의존하는지 코드에서 명확히 드러난다는 장점이 있다.

3.6. I/O와 연산의 결합 문제 → cache 계층 분리, disk cache, timing decorator

7.2.6에서 확인한 문제는 이미지 로드, latent 저장, CSV 기록, progress 저장 같은 I/O 작업과 GPU inference, metric 계산 같은 연산 작업이 하나의 루프 안에 결합되어 있다는 점이었다. 이 구조에서는 전체 실행 시간이 길어졌을 때 병목이 image I/O인지, encoder forward인지, latent 저장인지 분리해서 확인하기 어렵다.

이를 해결하기 위해 I/O와 연산 단계를 분리하고, cache 계층을 명시하였다. latent path가 존재하면 먼저 .npy 파일을 로드하고, 존재하지 않을 때만 이미지를 로드해 encoder를 실행한다. 즉, 저장된 latent 파일을 단순 output이 아니라 disk cache로 활용한다.

또한 timing decorator를 적용하여 주요 단계의 실행 시간을 측정한다. decorator는 실행 시간 측정이라는 부가 관심사를 핵심 연구 로직에서 분리한다. 이때 functools.wraps를 사용하여 wrapper 적용 후에도 원래 함수의 이름과 metadata가 유지되도록 한다.

이 개념은 decorator, closure, wrapper, functools.wraps, logging/timing decorator와 연결된다. 핵심 latent 분석 로직은 그대로 유지하면서, 시간 측정 정책을 외부에서 감싸는 방식으로 구조를 개선한다.

3.7. Resume 및 progress 처리의 구조적 한계 → set 기반 key 관리

7.2.7에서 분석한 문제는 기존 resume 방식이 processed_pairs라는 index에 의존한다는 점이었다. 이 방식은 valid_rows의 순서가 이전 실행과 동일하다는 가정이 필요하다. metadata CSV, variant 목록, missing file 상태가 바뀌면 같은 index가 같은 pair를 의미하지 않을 수 있다.

이를 해결하기 위해 (`class_id`, `image_id`, `variant`) tuple을 pair key로 사용하고, 이미 처리된 key를 `set`에 저장하는 방식을 적용하였다. 결과 CSV에 이미 존재하는 pair key를 읽어 `done_keys` set으로 만들고, 재실행 시 해당 key가 set에 있으면 건너뛴다.

이 방식에서 `set`을 사용하는 이유는 처리 완료 여부를 빠르게 검사해야 하기 때문이다. pair key가 이미 처리되었는지 매번 list에서 찾으려면 pair 수가 많아질수록 탐색 비용이 증가한다. 반면 `set`은 membership test에 적합한 자료구조이므로 resume 안정성과 처리 효율을 동시에 개선할 수 있다.

문제점과 적용 개념의 대응 요약 : 기존 코드의 문제점과 적용한 수업 개념은 다음과 같이 대응된다.

7.2 문제점	적용한 수업 개념	적용 이유
중복 original encoding	<code>dict</code> cache, disk cache, PyTorch inference wrapper	같은 original latent를 반복 계산하지 않고 재사용하기 위해
pair list materialization	iterator, generator, lazy evaluation	전체 pair list를 먼저 만들지 않고 필요한 시점에 순차 처리하기 위해
전역 설정값	<code>dataclass</code> , config/state 분리	실험 조건을 명시화하고 benchmark와 재현성을 높이기 위해
<code>main()</code> 책임 과다	단일책임원칙, 역할 분리	모델 로드, pair 생성, metric 계산, 저장, progress 관리를 분리하기 위해
metric 계산 중복	중간값 재사용, 계산 구조 명시화	norm, dot product를 재사용하여 metric 계산을 명확히 하기 위해
I/O와 연산 결합	disk cache, timing decorator, logging 분리	병목을 단계별로 측정하고 저장된 latent를 실제 cache로 활용하기 위해
index 기반 resume	<code>set</code> 기반 pair key 관리	row 순서가 바뀌어도 처리 완료 pair를 안정적으로 식별하기 위해

4. 개선 과정

4.1. 중복 original encoding 개선 : original latent cache 도입

- 수정 전 : `valid_rows`를 순회하면서 매 pair마다 original image와 variant image를 모두 로드하고 각각 encoding

```
for row in valid_rows:
    original_img = load_pil_rgb(row["original_path"])
    variant_img = load_pil_rgb(row["variant_path"])

    z_orig = encode_image(encoder, transform, original_img, device)
    z_var = encode_image(encoder, transform, variant_img, device)
```

이 구조에서는 같은 original image가 여러 variant와 비교될 때 original latent가 반복 계산된다. 이를 개선하기 위해 original latent cache를 도입하였다. 수정 후에는 (`class_id`, `image_id`)를 key로 사용하여 original latent를 memory cache에 저장하고, 이미 계산된 original latent는 다시 encoder를 호출하지 않고 cache에서 가져온다.

- 수정 후 : original image의 처리 단위를 pair가 아니라 image로 바꿈

```
original_cache = {}

def get_original_latent(config, encoder, transform, row, device):
    key = (row["class_id"], row["image_id"])

    if key in original_cache:
        return original_cache[key]

    path = latent_path_for(config, row["class_id"], row["image_id"], "original")

    if path.exists():
        z = torch.from_numpy(np.load(path)).float()
    else:
        image = load_pil_rgb(row["original_path"])
        z = encode_image(encoder, transform, image, device)
        if config.save_latents:
            save_latent(path, z)

    original_cache[key] = z
    return z
```

original image는 여러 variant와 비교되지만 latent 자체는 동일하므로 한 번만 계산해도 된다. 따라서 original encoding 횟수는 기존의 **valid pair** 수에서 **unique original image** 수에 가까운 값으로 줄어든다.

4.2. Pair list materialization 개선: **build_pair_rows()**를 generator로 변경

- 수정 전 : **build_pair_rows()**는 모든 pair를 list에 저장한 뒤 반환

```
rows = []

for _, r in df.iterrows():
    for variant in VARIANTS:
        rows.append({
            "class_id": class_id,
            "image_id": image_id,
            "variant": variant, ... })

return rows
```

이후 **main()**에서 다시 다음과 같이 valid/missing list 생성

```
rows = build_pair_rows()
valid_rows = [r for r in rows if r["original_exists"] and r["variant_exists"]]
missing_rows = [r for r in rows if not (r["original_exists"] and r["variant_exists"])]
```

- 수정 후 : pair row를 한 번에 모두 list로 반환하지 않고, generator로 하나씩 생산하도록 변경

```
def iter_pair_rows(config):
    df = pd.read_csv(config.meta_path)
    df = df[df["n_fg_labels"] == 1].copy()
    df = df.sort_values(["class_id", "image_id"]).reset_index(drop=True)

    if config.limit_images is not None:
        df = df.head(config.limit_images).copy()

    for _, r in df.iterrows():
        class_id = str(r["class_id"])
        image_id = str(r["image_id"])
        original_path = Path(str(r["original_path"]))

        for variant in config.variants:
            variant_path = variant_path_for(config, class_id, image_id, variant)

            yield {
                "class_id": class_id,
                "image_id": image_id,
                "variant": variant,
                "original_path": str(original_path),
                "variant_path": str(variant_path),
                "original_exists": original_path.exists(),
                "variant_exists": variant_path.exists(), }

    ...
```

분석 루프에서는 generator 직접 소비 :

```
for row in iter_pair_rows(config):
    if not (row["original_exists"] and row["variant_exists"]):
        write_missing_row(row)
        continue

    ...
```

이렇게 변경하면 전체 pair list와 valid/missing list를 모두 메모리에 들고 있을 필요가 줄어들며, pair 생성과 pair 처리가 더 느슨하게 연결되어 추후에 특정 variant만 분석하거나 일부 class만 분석하는 등의 조건을 추가하기도 쉬워진다.

4.3. 전역 설정값 개선 : LatentAnalysisConfig 도입

- 수정 전 : 실험 설정을 전역 상수로

```

META_PATH = Path(...)
VARIANT_ROOT = Path(...)
OUT_DIR = Path(...)
DEVICE = "cuda"
RESIZE_SIZE = 256
CROP_SIZE = 224
SEED = 42
SAVE_LATENTS = True

```

- 수정 후
 - 실험 설정을 하나의 dataclass로 묶음

```

from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True)
class LatentAnalysisConfig:
    meta_path: Path
    variant_root: Path
    out_dir: Path
    external_repo_root: Path
    checkpoint_path: Path
    arch: str = "vgg16"
    device: str = "cuda"
    resize_size: int = 256
    crop_size: int = 224
    seed: int = 42
    save_latents: bool = True
    limit_images: int | None = None

```

- 기존에 전역 변수로 의존하던 함수들은 config를 인자로 받도록 수정

```

def latent_path_for(config, class_id, image_id, condition):
    return config.out_dir / "latents" / class_id / f"{image_id}_{condition}_latent.npy"

```

이러면 실험 조건을 명시적으로 관리하기 쉬워진다. benchmark를 수행할 때는 `limit_images`, `out_dir`, `save_latents` 등을 다르게 설정한 config를 사용할 수 있고, 실제 전체 실험에서는 전체 데이터용 config를 사용할 수 있다.

4.4. `main()` 함수 책임 과다 개선: 역할별 함수 분리

- 수정 전 : `main()` 함수가 전체 파이프라인을 거의 모두 담당

```

set_seed(config.seed)
build_eval_transform(config)
load_encoder_from_imagenet_ae(config)
load_done_keys(meta_csv)
iter_pair_rows(config)
get_original_latent(...)
get_variant_latent(...)
compute_metrics(...)
write_metric_row(...)
summarize(...)

```

- 수정 후 : `main()` 또는 `run_pipeline()`이 세부 작업을 직접 수행하지 않고, 역할별 함수들을 호출하는 orchestration 함수가 되도록 변경, `run_pipeline()`의 핵심 흐름을 단순화함

```

def run_pipeline(config):
    prepare_output_dirs(config)
    set_seed(config.seed)

    device = resolve_device(config.device)
    transform = build_eval_transform(config.resize_size, config.crop_size)
    encoder = load_encoder_from_imagenet_ae(config, device)

    done_keys = load_done_keys(metric_csv_path(config))

    for row in iter_pair_rows(config):
        key = make_pair_key(row)

        if key in done_keys:
            continue

        if not is_valid_pair(row):
            write_missing_row(row)
            continue

        z_orig = get_original_latent(config, encoder, transform, row, device)
        z_var = get_variant_latent(config, encoder, transform, row, device)

        metrics = compute_metrics(z_orig, z_var)
        write_metric_row(row, z_orig, z_var, metrics)
        done_keys.add(key)

    summarize(config)

```

각 함수의 역할을 명확히 한다. metric 계산을 수정하고 싶으면

`compute_metrics()`만 확인하면 되고, cache 정책을 수정하고 싶으면

`get_original_latent()` 또는 `get_variant_latent()`만 확인하면

된다. 이로써 기존의 거대한 `main()` 함수보다 유지보수성과 테스트 가능성을 높인다.

4.5. Metric 계산 중복 개선 : dot product와 norm 재사용

- 수정 전 : `compute_metrics()`는 norm을 별도로 계산, cosine similarity는 `F.cosine_similarity()`로 계산

```
norm_original = torch.linalg.norm(z_orig).item()
norm_variant = torch.linalg.norm(z_var).item()
cos = F.cosine_similarity(z_orig.unsqueeze(0), z_var.unsqueeze(0), dim=1).item()
```

- 수정 후 : dot product와 norm을 명시적으로 계산하고, 이를 cosine similarity, cosine distance, angle 계산에 함께 사용하도록 변경

```
def compute_metrics(z_orig, z_var):
    z_orig = z_orig.float().flatten()
    z_var = z_var.float().flatten()

    diff = z_var - z_orig
    l2 = torch.linalg.norm(diff).item()

    norm_original = torch.linalg.norm(z_orig).item()
    norm_variant = torch.linalg.norm(z_var).item()

    dot = torch.dot(z_orig, z_var).item()
    denom = max(norm_original * norm_variant, 1e-12)

    cos = dot / denom
    cos = float(np.clip(cos, -1.0, 1.0))

    return {
        "l2": float(l2),
        "cosine_similarity": float(cos),
        "cosine_distance": float(1.0 - cos),
        "angle_radian": float(np.arccos(cos)),
        "norm_original": float(norm_original),
        "norm_variant": float(norm_variant),
        "norm_difference": float(norm_variant - norm_original),
        "relative_l2": float(l2 / max(norm_original, 1e-12)), }
```

이로써 metric 계산의 중복 가능성을 줄이고, 각 metric이 어떤 중간값을 기반으로 계산되는지를 명확히 보인다.

4.6. I/O와 연산 결합 개선 : latent load/save와 encoding 정책 분리

- 수정 전 : 루프 안에서 이미지 로드, encoder forward, latent 저장, CSV 기록을 연속적으로 수행함. original latent 파일이 이미 존재해도 계산 단계에서는 이를 재사용하지 않고, 저장할 때만 중복 저장을 피함.

```
z_orig = encode_image(encoder, transform, original_img, device)

if SAVE_LATENTS:
    if not orig_latent_path.exists():
        save_latent(orig_latent_path, z_orig)
```

`orig_latent_path`가 존재하더라도 original image는 이미 다시 encoding된 뒤라, 저장 파일이 cache로 작동하지 않는다.

- 수정 후 : latent를 가져오는 정책을 별도 함수로 분리

```
def load_or_encode_latent(path, image_path, encoder, transform, device, save=True):
    if path.exists():
        return torch.from_numpy(np.load(path)).float()

    image = load_pil_rgb(image_path)
    z = encode_image(encoder, transform, image, device)

    if save:
        save_latent(path, z)

    return z
```

latent 파일이 있으면 disk cache에서 로드하고, 없으면 이미지를 encoding한 뒤에 저장한다. 이를 통해 I/O와 연산의 관계가 명확해지고, 저장된 `.npy` 파일을 실제 cache로 활용할 수 있다.

4.7. Resume/progress 개선: index 기반에서 key 기반으로 변경

- 수정 전 : Resume은 `progress.json`의 `processed_pairs` index를 기준으로 작동함

```
if idx < start_idx: continue
```

`valid_rows`의 순서가 변하지 않는다는 가정에 의존한다.

- 수정 후 : (`class_id`, `image_id`, `variant`)를 pair key로 만들고, 이미 처리된 key를 set으로 관리하도록 변경함

```
def make_pair_key(row):
    return (row["class_id"], row["image_id"], row["variant"])

done_keys = load_done_keys(meta_csv)

for row in iter_pair_rows(config):
    key = make_pair_key(row)

    if key in done_keys: continue

    ...
    writer.writerow(...)
    done_keys.add(key)
```

이 변경을 통해 row 순서가 바뀌어도 이미 처리한 pair를 안정적으로 식별할 수 있으며, 실패한 pair는 별도 CSV에 기록하여 추후 재처리하거나 오류 원인을 분석할 수 있게 하였다.

4.8. Timing decorator 추가: benchmark와 병목 분석 지원

- 수정 전 : 각 단계의 실행 시간을 독립적으로 측정하기 어려워 전체 실행 시간이 길어졌을 때 encoder 로드, pair 처리, metric 계산, summary 생성 중 어느 부분이 병목인지 확인하기 어렵다.
- 수정 후 : timing decorator 추가


```

from functools import wraps
from time import perf_counter

def timed(name):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):
            start = perf_counter()
            result = fn(*args, **kwargs)
            elapsed = perf_counter() - start
            print(f"[TIME] {name}: {elapsed:.4f}s", flush=True)
            return result
        return wrapper
    return decorator

```

이 decorator는 `load_encoder_from_imagenet_ae`, `run_pipeline`, `summarize` 등에 적용하였으며, 이를 통해 시간 측정 로직이 함수 내부에 들어가지 않고, `wrapper`를 통해 일관되게 적용된다.

4.9. 고려했지만 적용하지 않은 대안

01. 모든 latent vector를 한 번에 tensor로 모아 GPU에서 pairwise metric을 matrix 연산으로 계산하는 방식을 고려하였다. 이 방식은 연산 속도 측면에서는 유리할 수 있지만, 전체 latent vector를 한 번에 메모리에 올려야 하므로 peak memory가 크게 증가할 수 있다. 또한 본 코드의 목적은 각 original-variant pair별 metric CSV와 latent 파일을 안정적으로 생성하는 것이므로, 전체 pairwise matrix 계산은 현재 목적에 비해 과도하다고 판단하였다.
02. 모든 original latent와 variant latent를 memory cache에 저장하는 방식도 고려하였다. 그러나 variant latent는 original latent처럼 여러 pair에서 반복 사용되는 구조가 아니므로 memory cache의 이점이 상대적으로 작다. 반면 모든 variant latent를 메모리에 보관하면 데이터 규모가 커질수록 메모리 사용량이 증가한다. 따라서 memory cache는 original latent 중심으로 제한하고, variant latent는 disk cache를 사용하는 방식으로 결정하였다.
03. multiprocessing 또는 multi-threading을 이용한 이미지 로딩 병렬화도 고려하였다. 그러나 GPU encoder forward, PIL image load, `.npy` 저장, CSV write가 함께 섞여 있는 상태에서 병렬화를 먼저 적용하면 병목 원인을 파악하기 어렵기 때문에 먼저 구조 분리와 caching을 적용하기로 하였다.

04. `torch.compile` 적용도 고려할 수 있다. 하지만 본 코드는 사전학습된 외부 Autoencoder encoder를 inference 목적으로 호출하는 구조이고, 전체 병목이 모델 연산뿐 아니라 이미지 I/O와 latent 저장에도 분산되어 있다. 따라서 이번 개선에서는 `torch.compile`을 적용하지 않았다.

4.10. 최적화 과정에서 발생한 trade-off

01. 속도와 메모리 사용량 사이의 균형

original latent를 memory cache에 저장하면 encoder 반복 호출과 disk load를 줄일 수 있어 실행 시간은 감소한다. 그러나 unique original image 수가 매우 많아지면 cache가 차지하는 메모리가 증가할 수 있다. 따라서 필요하다면 cache 크기를 제한하거나, memory cache를 끄고 disk cache만 사용하는 옵션을 둘 수 있다.

02. lazy evaluation과 실험 기록 사이의 균형

pair row를 generator로 처리하면 중간 list 생성을 줄일 수 있지만, 전체 pair 목록을 `pairs_all.csv`로 저장하려면 generator를 별도로 순회하거나 처리 루프에서 동시에 기록해야 한다. 따라서 분석 루프는 lazy하게 유지하되, 실험 재현을 위한 metadata 기록은 별도 writer를 통해 보존하는 방식을 선택하였다.

03. 구조 분리와 코드 복잡도 사이의 균형

기존 코드는 하나의 스크립트에 모든 흐름이 들어 있어 실행은 간단하다. 반면 개선 후에는 config, cache, pair generator, metric, progress, writer 함수가 분리되므로 코드 길이와 함수 수는 증가한다. 하지만 연구 코드가 반복적으로 재실행되고, benchmark와 조건 변경이 필요하다는 점을 고려하면 구조적 명확성과 유지보수성의 이점이 더 크다고 판단하였다.

4.11. 개선 과정 요약

정리하면, 7.2에서 분석한 문제점에 맞춰 다음과 같이 개선하였다. 중복 original encoding 문제는 dict 기반 original latent cache로 해결하고, pair list materialization 문제는 generator 기반 pair row 생산으로 줄였다. 전역 설정값 문제는 dataclass 기반 config로 정리했으며, `main()` 함수의 책임 과다는 역할별 함수 분리와

단일책임원칙 적용으로 개선하였다. Metric 계산 중복은 dot product와 norm 중간값 재사용으로 줄였고, I/O와 연산 결합 문제는 latent load/save 정책 분리와 disk cache 활용으로 완화하였다. 마지막으로 resume/progress 처리는 index 기반 방식에서 set 기반 pair key 방식으로 변경하여 안정성을 높였다.

이러한 개선은 단순한 코드 정리가 아니라, Sphere 프로젝트의 AE latent distance·angle 분석 파이프라인을 더 빠르고, 재현 가능하며, 확장 가능한 구조로 바꾸기 위한 것이라 할 수 있다.

5. 최적화 결과

01. 수정 후 코드에 반영된 핵심 변경

2. 문제점	코드 반영
중복 original encoding	<code>original_cache</code> 와 <code>get_original_latent()</code> 추가
pair list materialization	<code>iter_pair_rows()</code> generator 추가
전역 설정값	<code>LatentAnalysisConfig</code> dataclass 추가
<code>main()</code> 책임 과다	<code>run_pipeline()</code> , <code>get_original_latent()</code> , <code>get_variant_latent()</code> , <code>summarize()</code> 등으로 분리
metric 계산 중복	<code>dot</code> , <code>norm_original</code> , <code>norm_variant</code> 중간값 재사용
I/O와 연산 결합	<code>load_latent()</code> , <code>save_latent()</code> , disk cache 정책 분리
resume/progress 한계	<code>(class_id, image_id, variant)</code> key 기반 <code>done_keys</code> 적용
benchmark 어려움	<code>run_stats.json</code> , CLI option, timing decorator 추가

02. 실험 조건 및 세팅

- 비교는 동일한 metadata, 동일한 variant 목록, 동일한 Autoencoder checkpoint, 동일한 device, 동일한 이미지 전처리 조건에서 수행하였다.
- 입력 이미지 수는 `LIMIT_IMAGES` 값을 기준으로 조절하였으며, 각 입력 크기에 대해 원본 코드와 개선 코드를 각각 실행하였다.

□ 비교 대상 :

□ Before: 기존 `ae_s919_original_variant_metrics.py`

□ After: original latent cache, generator 기반 pair 처리, dataclass config, key 기반 resume, disk cache, timing decorator를 적용한 개선 코드

□ After Cold : 최적화 코드를 latent cache가 없는 상태에서 처음 실행

□ After Warm : 동일 output directory에 이미 저장된 latent `.npy` 파일이 존재하는 상태에서 최적화 코드를 재실행

□ 실험 조건 :

항목	값
Dataset	ImageNetS919 single-object images
Variant 종류	alpha, bg_color, bg_inpaint, bg_transparent, comp_black, comp_white, cutout_rgba, gaussian, masked_black, masked_white, salt_pepper, mask_white
Autoencoder checkpoint	imagenet-vgg16.pth
Input transform	Resize 256, CenterCrop 224, ToTensor
Device	CUDA 또는 CPU
Random seed	42
입력 크기	10, 30, 50, 100 images
측정 항목	실행 시간, peak memory, pair 수, encoding 횟수, 코드 구조 변화

공정한 비교를 위해 각 입력 크기별로 별도의 output directory를 사용하였다. 또한 이미 처리된 pair를 건너뛰는 resume 기능이 실행 시간에 영향을 주지 않도록, 비교 실험에서는 새로운 output directory를 사용하거나 `--no-resume` 옵션을 사용하였다.

03. 실행 시간 비교 : `/usr/bin/time -v`를 사용하여 측정

- After 코드의 encoding 횟수와 cache hit 수 : `run_stats.json`에 기록된 값 사용

입력 이미지 수	Before 실행 시간 (s)	After cold 실행 시간(s)	After warm 실행 시간(s)
10	8.4	9.54	4.96
30	13.9	17.45	5.12
50	19.62	25.27	5.24
100	35.34	46.76	5.58

After cold는 모든 입력 크기에서 before보다 느리게 측정되었다. 이는 개선 코드에서 original latent cache, disk cache 확인, key 기반 resume 관리, config 저장, run statistics 기록 등 추가적인 bookkeeping 작업이 수행되기 때문이다. 또한 cold run에서는 variant image의 latent가 아직 저장되어 있지 않으므로, variant encoding은 기존 코드와 동일하게 수행된다.

반면 after warm은 모든 입력 크기에서 before보다 빠르게 측정되었다. 특히 입력 이미지 100개 조건에서는 before가 35.34초, after warm이 5.58초로 측정되어 약 84.2%의 실행 시간 감소를 보였다. 이는 저장된 .npy latent 파일을 disk cache로 재사용하여 encoder forward를 거의 수행하지 않았기 때문이다.

입력 이미지 수	Before 대비 After warm 실행 시간 감소율
10	약 41.0%
30	약 63.2%
50	약 73.3%
100	약 84.2%

이 결과는 개선 코드의 장점이 단순한 최초 실행 시간 감소보다는, latent가 한 번 생성된 이후 반복 분석 또는 metric 재계산 상황에서 크게 나타난다는 것을 보여준다.

04. 메모리 사용량 비교 : /usr/bin/time -v의 Maximum resident set size 값을 MB 단위로 변환하여 비교

입력 이미지 수	Before peak memory(MB)	After cold peak memory(MB)	After warm peak memory(MB)
10	885.80	873.54	801.21
30	851.73	903.31	816.67

50	889.25	881.67	819.31
100	872.27	889.88	819.27

After cold의 메모리 사용량은 before보다 항상 감소하지는 않았다. pair row를 generator로 처리하여 중간 list materialization을 줄였지만, original latent memory cache, 처리 완료 key 관리, 실행 통계 저장 등의 추가 구조가 생겼기 때문이다. 따라서 cold run의 메모리 결과는 pair list 제거로 인한 감소 효과와 cache 관리 구조로 인한 증가 효과가 함께 반영된 것으로 해석할 수 있다.

반면 after warm에서는 모든 입력 크기에서 peak memory가 before보다 낮게 측정되었다. Warm run에서는 latent vector를 encoder로 새로 생성하지 않고 disk cache에서 로드하여 metric 계산만 수행하므로, GPU inference와 관련된 일시적 메모리 사용이 줄어든 것으로 볼 수 있다.

05. 중복 original encoding 제거 결과

개선 코드의 가장 명확한 효과는 original latent vector의 중복 encoding 제거이다.

입력 이미지 수	valid pair 수	Before original encoding	After cold original encoding	After warm original encoding
10	120	120	10	0
30	360	360	30	0
50	600	600	50	0
100	1200	1200	100	0

기존 코드는 original-variant pair를 처리 단위로 삼았기 때문에, 하나의 original image가 12개 variant와 비교될 경우 original latent도 12번 반복 계산되었다. 반면 after cold에서는 (class_id, image_id) 기반 original latent cache를 사용하여 original image당 한 번만 encoding하였다. 따라서 original encoding 횟수는 valid pair 수가 아니라 unique original image 수에 비례하게 되었다.

예를 들어 입력 이미지 100개 조건에서 before는 original latent를 1200회 계산하지만, after cold는 100회만 계산한다. 이는 original encoding 기준으로 약 91.7%의 중복 계산이 제거되었음을 의미한다.

After warm에서는 이미 저장된 original latent 파일을 disk cache로 로드하므로 original encoding 횟수가 0회가 되었다.

06. Variant encoding과 disk cache 효과

Variant image는 각 variant마다 실제 이미지가 다르므로, after cold에서는 variant encoding 횟수가 before와 동일하다. 그러나 after warm에서는 저장된 variant latent 파일을 disk cache로 재사용할 수 있다.

입력 이미지 수	Before Variant encoding	After Cold Variant encoding	After Warm Variant encoding	After Warm Variant disk hits
10	120	120	0	120
30	360	360	0	360
50	600	600	0	600
100	1200	1200	0	1200

이 결과는 개선 코드가 저장된 latent를 단순한 산출물이 아니라 실제 계산 cache로 활용한다는 점을 보여준다. 기존 코드에서는 latent 파일을 저장하더라도 계산 전에 이를 로드하지 않았기 때문에, 재실행 시 encoder forward를 충분히 줄이지 못한다. 반면 개선 코드에서는 latent 파일이 존재하면 먼저 `.npy`를 로드하고, 파일이 없을 때만 이미지를 encoder에 통과시킨다.

따라서 after warm 조건에서는 original과 variant 모두 encoder forward 없이 disk cache에서 latent를 불러와 metric 계산을 수행할 수 있었다.

07. 입력 크기별 성능 변화

입력 이미지 수가 증가하면 pair 수는 이미지 수와 variant 수의 곱에 비례하여 증가한다. 본 실험에서는 variant 수가 12개이므로, 입력 이미지 수 `N`에 대해 valid pair 수는 `12N`으로 증가하였다.

Before와 after cold는 모두 입력 이미지 수가 증가할수록 실행 시간이 증가하였다. 그러나 after warm의 실행 시간은 10개 이미지에서 4.96초, 100개 이미지에서 5.58초로 증가 폭이 작았다.

이는 encoder forward를 수행하지 않고 저장된 latent를 disk cache로 로드했기 때문이다. 즉, after warm에서는 전체 시간이 이미지 encoding보다 CSV 처리, .npy 로드, metric 계산 같은 상대적으로 가벼운 작업에 의해 결정된다.

이 결과는 개선 코드가 최초 실행보다 후속 재분석 상황에서 더 큰 성능 이점을 갖는다는 점을 보여준다. 예를 들어 metric 계산 방식을 바꾸거나, summary 방식만 바꾸거나, 동일 latent에 대해 다른 분석을 수행하는 경우에는 이미 저장된 latent를 재사용할 수 있으므로 전체 실행 시간을 크게 줄일 수 있다.

08. 코드 구조 변화 : 성능 수치와 별도로, 코드 구조는 다음과 같이 개선되었다.

비교 항목	Before	After
설정 관리	전역 상수	LatentAnalysisConfig
pair 생성	전체 list 생성	generator 기반 순차 생성
original latent	매 pair마다 encoding	image 단위 memory/disk cache
variant latent	매 실행마다 encoding	disk cache 재사용 가능
저장된 latent	후속 분석용 산출물	계산 전 재사용 가능한 cache
resume 방식	index 기반	(class_id, image_id, variant) key 기반
metric 계산	F.cosine_similarity() 내부 계산 의존	dot/norm 중간값 명시적 재사용
시간 측정	별도 구조 없음	timing decorator 및 run statistics
전체 구조	main() 중심 단일 흐름	역할별 함수 분리

이 구조 변화는 실행 시간 개선뿐 아니라 연구 코드의 재사용성, 실험 조건 변경 용이성, 재현성 측면에서도 의미가 있다.

09. 실험 조건 변경 시 코드 수정 범위 비교

기존 코드에서는 실험 조건을 바꾸기 위해 스크립트 상단의 전역 상수를 직접 수정해야 했다. 반면 개선 코드에서는 command-line argument와 config 객체를 통해 실행 조건을 변경할 수 있다.

따라서, 개선 코드는 단순히 계산 코드만 바꾼 것이 아니라, 반복 실험을 수행하기 쉬운 형태로 재구성한 것이다.

변경 상황	Before	After
입력 이미지 수 변경	<code>LIMIT_IMAGES</code> 전역 상수 수정	<code>--limit-images</code> 인자 변경
output directory 변경	<code>OUT_DIR</code> 전역 상수 수정	<code>--out-dir</code> 인자 변경
특정 variant만 실행	<code>VARIANTS</code> list 직접 수정	<code>--variants</code> 인자 사용
resume 끄기	코드 수정 필요	<code>--no-resume</code> 인자 사용
disk cache 끄기	코드 수정 필요	<code>--no-disk-cache</code> 인자 사용

10. 최적화 결과 요약

본 최적화의 결과는 cold run과 warm run에서 다르게 나타났다. Cold run에서는 after 코드가 before보다 느리게 측정되었다. 이는 cache 확인, 통계 기록, config 저장, key 기반 resume 관리 등 구조 개선을 위한 부가 overhead가 추가되었고, variant encoding은 여전히 수행되었기 때문이다.

그러나 중복 original encoding 제거 효과는 명확했다. After cold에서 original encoding 횟수는 valid pair 수에서 unique original image 수로 감소하였고, 본 실험에서는 약 91.7%의 original encoding 중복이 제거되었다.

또한 warm run에서는 저장된 original 및 variant latent를 disk cache로 재사용하여 encoder forward를 수행하지 않았고, 그 결과 모든 입력 크기에서 before보다 빠른 실행 시간을 보였다. 특히 입력 이미지 100개 조건에서는 before 대비 약 84.2%의 실행 시간 감소가 나타났다.

따라서 본 개선은 최초 실행 속도를 무조건 빠르게 만드는 최적화라기보다, Sphere 프로젝트처럼 동일한 latent representation을 반복적으로 분석하거나 metric 계산 방식을 바꾸어 재실행하는 연구 환경에서 효과적인 cache-aware pipeline 개선이라고 할 수 있다.

6. 결과 해석 및 한계

본 절에서는 7.5의 benchmark 결과를 바탕으로, 최적화가 기대한 방식으로 작동했는지, 어떤 조건에서 효과가 컸는지, 그리고 어떤 한계가 있었는지를 분석한다. 본 실험에서는 before, after cold, after warm 세 조건을 비교하였다. Before는 기존 코드, after cold는 latent cache가 없는 상태에서 최적화 코드를 처음 실행한 조건, after warm은 동일 output directory에 이미 저장된 latent .npy 파일이 존재하는 상태에서 최적화 코드를 재실행한 조건이다.

6.1. 기대한 개선이 실제로 나타났는지

본 최적화에서 가장 기대했던 개선은 동일한 original image의 latent vector가 variant 수만큼 반복 encoding되는 문제를 제거하는 것이었다. 이 부분은 실제 결과에서 명확하게 확인되었다. 입력 이미지 수가 10, 30, 50, 100개일 때 valid pair 수는 각각 120, 360, 600, 1200개였고, 기존 코드에서는 original encoding 횟수도 valid pair 수와 동일하게 증가하였다. 반면 after cold에서는 original encoding 횟수가 각각 10, 30, 50, 100회로 감소하였다.

즉, original latent encoding 횟수는 $N \times V$ 에서 N 으로 감소하였다. 본 실험에서는 variant 수가 12개였으므로, original encoding 기준으로 약 91.7%의 중복 계산이 제거되었다. 이는 (class_id, image_id) 기반 original latent cache가 의도한 대로 작동했음을 보여준다.

그러나 전체 실행 시간 관점에서는 기대한 결과가 cold run에서 바로 나타나지는 않았다. After cold는 모든 입력 크기에서 before보다 느렸다. 예를 들어 입력 이미지 100개 조건에서 before는 35.34초, after cold는 46.76초로 측정되었다. 이는 original encoding을 줄였음에도 불구하고, 개선 코드가 추가적인 cache 확인, key 관리, config 저장, statistics 기록, disk cache path 검사 등을 수행하기 때문이다. 따라서 본 최적화는 “최초 실행 시간을 무조건 줄이는 최적화”라기보다는, “중복 계산을 제거하고 재실행 가능한 cache-aware pipeline으로 구조를 바꾸는 최적화”라고 해석하는 것이 적절하다.

반면 after warm 조건에서는 기대한 실행 시간 개선이 뚜렷하게 나타났다. 입력 이미지 100개 조건에서 after warm은 5.58초로 측정되어 before 대비 약 84.2%의 실행 시간 감소를 보였다. Warm run에서는 original과 variant latent가 모두 disk cache에서 로드되었고, original_encoded와 variant_encoded가 모두 0회로 기록되었다. 이 결과는 저장된 latent 파일이 단순한 산출물이 아니라 실제 계산 cache로 활용될 수 있음을 보여준다.

6.2. 개선 폭이 큰 부분과 그렇지 않은 부분

□ 개선 폭이 가장 큰 부분 : original encoding 횟수와 warm run 실행 시간

Original encoding은 before에서는 valid pair 수에 비례했지만, after cold에서는 unique original image 수에 비례하였다. 따라서 입력 이미지 수와 관계없이 original encoding 중복 제거율은 약 91.7%로 일정하게 나타났다.

- 또한 after warm에서는 original뿐 아니라 variant latent도 disk cache에서 로드되었기 때문에 encoder forward가 사실상 제거되었다. 이로 인해 입력 이미지 수가 증가해도 실행 시간이 크게 증가하지 않았다. 예를 들어 before는 입력 이미지 수가 10개에서 100개로 증가할 때 8.40초에서 35.34초로 증가했지만, after warm은 4.96초에서 5.58초로만 증가하였다. 이는 warm run의 병목이 더 이상 encoder forward가 아니라 .npy 파일 로드, metric 계산, CSV 기록 같은 상대적으로 가벼운 작업으로 이동했기 때문이다.

□ 개선 폭이 크지 않았거나 부정적으로 나타난 부분 : after cold의 전체 실행 시간

After cold는 original encoding을 줄였지만 variant encoding은 그대로 수행해야 한다. Variant image는 서로 다른 변형 이미지이므로, 첫 실행에서는 각 variant latent를 새로 계산해야 한다. 따라서 전체 encoder 호출 중 상당 부분이 여전히 남아 있다. 여기에 구조 개선을 위한 추가 overhead가 더해지면서 cold run의 wall-clock time은 before보다 길어졌다.

- 메모리 사용량도 일관된 개선을 보이지 않았다.

After cold에서는 pair row materialization을 줄였지만, original latent cache와 run statistics, key 관리 구조가 추가되었다. 그 결과 입력 크기에 따라 before보다 메모리가 줄어든 경우도 있고 증가한 경우도 있었다. 반면 after warm에서는 모든 입력 크기에서 before보다 낮은 peak memory를 보였는데, 이는 encoder forward가 수행되지 않아 inference 과정의 임시 메모리 사용이 줄었기 때문으로 해석된다.

6.3. 성능은 좋아졌지만 코드 복잡도가 증가한 부분

개선 코드에서는 original cache, disk cache, dataclass config, key-based resume, generator 기반 pair 처리, timing decorator, run statistics 기록이 추가되었다. 이로 인해 연구 파이프라인의 재사용성과 재현성은 증가했지만, 코드 복잡도도 함께 증가하였다.

기존 코드는 하나의 `main()` 함수 안에서 위에서 아래로 실행되는 단순한 절차적 구조였다. 따라서 빠르게 읽고 실행하기에는 편리했다. 반면 개선 코드는 `LatentAnalysisConfig`, `iter_pair_rows()`, `get_original_latent()`, `get_variant_latent()`, `load_or_encode_latent()`, `compute_metrics()`, `run_pipeline()` 등 여러 함수로 역할이 분리되었다. 이 구조는 유지보수성과 테스트 가능성을 높이지만, 처음 코드를 읽는 사람에게는 전체 흐름을 파악하기 위해 더 많은 함수 간 관계를 이해해야 한다는 부담이 생긴다.

특히 cache 정책은 코드 복잡도를 증가시키는 주요 요인이다. Memory cache와 disk cache를 모두 고려해야 하며, cold run과 warm run의 동작이 서로 다르다. 또한 resume 기능을 key 기반으로 바꾸면서 이미 처리된 pair를 식별하는 로직도 추가되었다. 이러한 복잡도는 반복 실험과 재실행이 많은 연구 환경에서는 가치가 있지만, 한 번만 실행하는 단순 스크립트에서는 과한 구조일 수 있다.

6.4. 메모리는 줄었지만 실행 시간이 늘어난 부분

After cold에서는 일부 입력 크기에서 peak memory가 before보다 낮게 측정되었지만, 실행 시간은 오히려 증가하였다. 이는 메모리 사용량과 실행 시간이 항상 같은 방향으로 개선되지 않는다는 점을 보여준다.

Pair row를 generator로 처리하면 `rows`, `valid_rows`, `missing_rows` 같은 중간 list materialization을 줄일 수 있다. 따라서 이론적으로는 메모리 사용량을 줄이는 데 도움이 된다. 그러나 generator는 매번 값을 하나씩 생성하고, 각 row에 대해 path 검사, key 생성, cache 조회, missing pair 기록 여부 판단 등의 로직을 수행한다. 즉, 한 번에 list를 만들어 순회하는 방식보다 Python-level overhead가 증가할 수 있다.

또한 after cold에서는 original latent cache를 유지한다. 이 cache는 original encoding을 줄이기 위한 구조이지만, unique original image 수가 많아질수록 일정한 메모리를 점유한다. 따라서 pair list 제거로 줄어드는 메모리와 cache로 증가하는 메모리가 동시에 존재한다. 본 실험에서 after cold의 메모리 사용량이 일관되게 감소하지 않은 것은 이러한 trade-off의 결과로 볼 수 있다.

결론적으로 after cold에서 일부 메모리 절감 가능성은 있었지만, cache 관리와 구조 분리에 따른 overhead 때문에 wall-clock time은 증가하였다. 이는 최적화에서 메모리 효율성과 실행 시간 효율성이 항상 동시에 개선되지는 않는다는 점을 보여준다.

6.5. Generator 기반 Lazy evaluation 적용으로 생긴 장단점

□ 장점 :

- 전체 pair list를 한 번에 만들지 않아도 된다는 것
기존 코드는 `build_pair_rows()`에서 모든 original-variant pair를 list에 저장하고, 이후 다시 `valid_rows`와 `missing_rows`로 분리하였다. 개선 코드에서는 pair row를 하나씩 생성하여 바로 처리할 수 있으므로, 중간 list materialization을 줄이고 데이터 흐름을 streaming 방식에 가깝게 만들 수 있다.
- 확장성 측면에서 유리함
특정 class만 처리하거나, 특정 variant만 처리하거나, 조건을 만족하는 row만 처리하는 정책을 추가하기 쉽다. 전체 pair list를 먼저 확정하지 않아도 되기 때문에, 데이터 규모가 더 커질 때도 처리 흐름을 유연하게 유지할 수 있다.

□ 단점 :

- 전체 pair 수를 사전에 쉽게 알기 어렵고, 전체 pair 목록을 `pairs_all.csv`로 저장하려면 generator를 순회하면서 별도 writer를 사용해야 한다.
- generator는 한 번 소비되면 다시 사용할 수 없으므로, 동일한 pair sequence를 여러 번 사용해야 하는 경우에는 다시 생성해야 한다. 이 점에서 디버깅이나 반복 분석에는 list materialization이 더 편리할 수 있다.

따라서 본 코드에서는 모든 단계를 lazy하게 만들지는 않았다.

Pair 처리 단계에는 lazy evaluation을 적용했지만, summary 계산 단계에서는 pandas DataFrame을 사용하여 전체 metric CSV를 eager하게 읽어들었다. Summary는 전체 결과에 대한 groupby aggregation이 필요하므로, 이 단계에서는 eager materialization이 더 적절하다.

6.6. Decorator와 class 구조 도입으로 생긴 오버헤드

- timing decorator와 dataclass config 구조 도입
: 코드 구조와 재현성 측면에서는 장점이 있지만, 실행 시간 관점에서는 약간의 overhead를 만든다.

□ Timing decorator

- 함수 실행 전후에 시간을 측정하고 로그를 출력한다.
- 이 overhead는 큰 함수에서는 작지만, 작은 함수에 반복적으로 적용되면 누적될 수 있다.
- 본 코드에서는 주로 `run_pipeline()`, `summarize()`처럼 큰 단위 함수에 적용했기 때문에 decorator 자체의 비용이 전체 성능을 크게 좌우하지는 않았을 것으로 보인다. 그러나 timing, logging, statistics 기록은 모두 cold run에서 추가 작업으로 작용한다.

□ Dataclass config

- 전역 상수보다 실험 조건을 명확하게 표현한다.
- `LatentAnalysisConfig`를 통해 metadata path, variant root, output directory, checkpoint path, device, resize size, crop size, variant 목록, cache 정책을 하나의 객체로 관리할 수 있다. 이는 benchmark와 재실행에는 유리하지만, 함수마다 config를 전달해야 하므로 코드가 길어지고 호출 구조가 복잡해진다.

즉, decorator와 class 구조는 계산 자체를 빠르게 하기 위한 장치라기보다, 측정 가능성, 재현성, 유지보수성을 높이기 위한 구조적 개선이다. 본 실험에서 after cold가 before보다 느렸던 이유 중 일부는 이러한 구조적 overhead와 관련이 있다.

6.7 추가로 개선 가능한 부분

01. batch 단위 encoder inference 적용 : 현재 코드는 이미지를 하나씩 로드하고 하나씩 encoder에 통과시킨다. PyTorch 모델은 일반적으로 batch inference에서 GPU 활용률이 더 높아질 수 있으므로, original image와 variant image를 batch로 묶어 처리하면 cold run의 실행 시간을 줄일 가능성이 있다.
02. PyTorch `DataLoader`를 활용한 병렬 image loading을 고려 가능 : 현재 pipeline에서는 PIL image open, resize, crop, tensor 변환이 반복적으로 수행된다. 이 과정이 I/O 병목이라면 `num_workers`를 사용하는 DataLoader 기반 구조가

도움이 될 수 있다. 다만 외부 Autoencoder repository와 현재 경로 구조를 Dataset class로 감싸야 하므로 추가 설계가 필요하다.

03. latent 저장 방식을 개선 가능 : 현재는 pair마다 `.npy` 파일을 개별 저장한다. 파일 수가 많아지면 파일 시스템 overhead가 증가할 수 있으므로, 하나의 HDF5, Zarr, Parquet, 또는 memory-mapped array 형태로 저장하는 방식을 고려할 수 있다. 특히 warm run에서 `.npy` 파일을 많이 로드할 때 파일 open overhead가 발생할 수 있다.
04. metric 계산을 vectorized 방식으로 변경 가능 : 현재는 pair별로 `compute_metrics()`를 호출한다. 동일한 original image에 대한 12개 variant latent를 한 번에 tensor로 쌓아 L2 distance와 cosine similarity를 batch 연산으로 계산하면 Python loop overhead를 줄일 수 있다.
05. cache 정책 선택 가능하게 변경 : 현재 after 코드는 memory cache와 disk cache를 모두 활용한다. 데이터 규모가 더 커질 경우 memory cache가 부담이 될 수 있으므로, `--cache-mode memory, --cache-mode disk, --cache-mode none` 같은 옵션을 제공하면 실험 환경에 따라 trade-off를 조절할 수 있다.
06. cold run에서 after가 느린 문제를 해결하기 위해 statistics 기록과 path existence check를 최소화 가능 : benchmark 목적이 아닐 때는 run statistics를 간소화하거나, latent 존재 여부를 사전에 index로 만들어 반복적인 `path.exists()` 호출을 줄일 수 있다.
07. 모델 연산 자체가 병목으로 확인될 경우 AMP 또는 `torch.compile`을 고려 가능 : 다만 본 실험에서는 cold run의 병목이 모델 연산에만 있는 것이 아니라 I/O와 bookkeeping에도 분산되어 있으므로, 먼저 profiler를 사용하여 실제 병목을 확인한 뒤 적용하는 것이 적절하다.

6.8 종합 해석

본 최적화는 모든 조건에서 단순히 실행 시간을 줄이는 방식으로 성공한 것은 아니다. After cold에서는 original encoding 중복 제거에도 불구하고 전체 실행 시간이 증가하였다. 이는 코드 구조 개선과 cache 관리에 따른 overhead, 그리고 variant encoding과 I/O가 여전히 남아 있었기 때문이다.

그러나 original encoding 중복 제거라는 목표는 명확하게 달성되었다. 또한 after warm에서는 저장된 latent를 disk cache로 재사용하여 original과 variant encoding을 모두 0회로 줄였고, 입력 이미지 100개 조건에서 before 대비 약 84.2%의 실행 시간 감소를 보였다. 따라서 개선 코드의 효과는 최초

실행보다 반복 실험, 후속 분석, metric 재계산, summary 재생성처럼 동일 latent representation을 재사용하는 연구 상황에서 크게 나타난다.

결론적으로 본 개선은 단순한 wall-clock time 최적화라기보다, Sphere 프로젝트의 AE latent 분석 코드를 cache-aware하고 재실행 가능한 연구 파이프라인으로 바꾼 구조적 최적화라고 해석할 수 있다.

